



UITs

Future will be better than thy past

An initiative of PHP Family

University of Information Technology & Sciences

Project Report

Course Code: CSE-360

Course Title: Microprocessor and Microcontrollers

Project Title: Project Name: Calculator For Simple
Mathematical Calculation

Submitted to:

Md. Ismail

Lecturer, UITs,

B.Sc. in CSE, RUET.

Submitted by:

Name : Md Hasnine Kabir

ID : 0432310005101017

Batch : 53rd

Section: 6A1

Date: 20/11/2025

Introduction

This project aims to design and implement a **simple mathematical calculation program** capable of performing basic arithmetic operations. The primary focus of the project is to understand how user input, logical processing, and output generation work together in a structured program.

The project has been developed for beginners to practice fundamental programming concepts such as flow control, modular design, function creation, and error handling.

A graphical or text-based interface (depending on implementation) allows the user to interact with the calculator easily and perform operations like **addition, subtraction, multiplication, and division**.

The implementation helps build a strong foundation in project structuring, code organization, and problem-solving using simple mathematical operations.

Method & Description

The calculator is designed using a simple interface that guides the user step-by-step. The program flow follows a systematic and modular approach to ensure clarity and ease of use.

1. User Interface (UI)

- A clean and simple interface displays a list of available operations.
- The user selects the desired operation from the menu.
- The program prompts the user to enter the required numbers.

2. Input Processing

- The program accepts numerical input from the user.
- Input validation ensures that only valid integers or floating-point numbers are considered.
- Any invalid input triggers an appropriate error message.

3. Core Functionalities

Each mathematical operation is implemented using a separate function or module for better modularity.

Supported operations:

- **Addition ($A + B$)**
- **Subtraction ($A - B$)**
- **Multiplication ($A \times B$)**

- **Division ($A \div B$)** with division-by-zero protection

4. Processing Logic

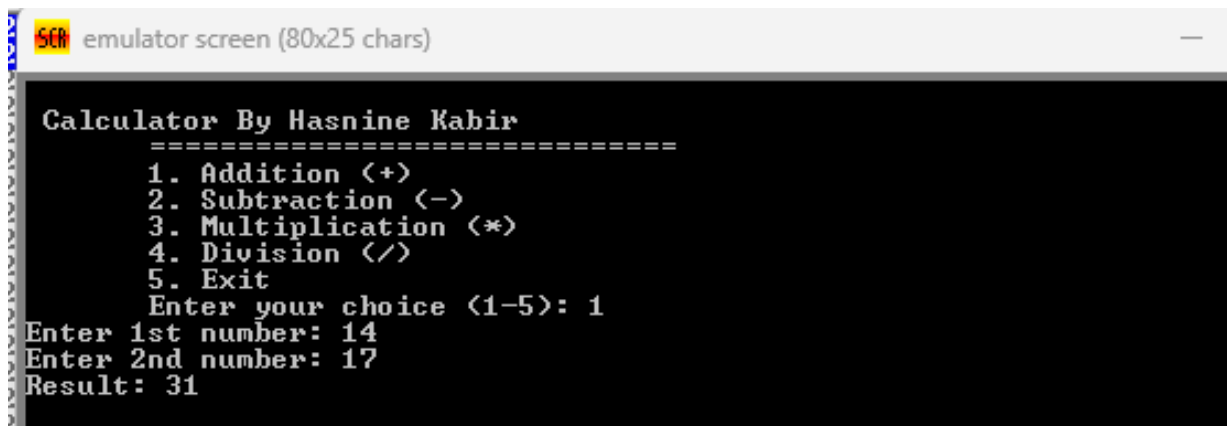
- The selected operation is applied to the user-provided numbers.
- The program handles special conditions such as:
 - Negative numbers
 - Large multi-digit input
 - Division by zero
- The result is computed and stored, and then printed back to the user.

5. Output Handling

- After computation, the final result is displayed clearly.
- The user is asked whether they want to perform more calculations or exit the program.

Sample Output

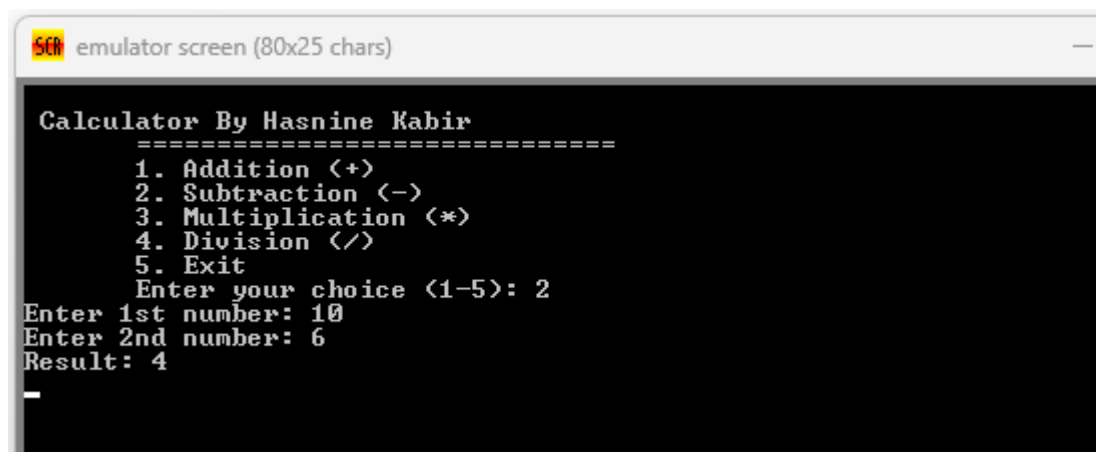
Example 1: Addition



```
emulator screen (80x25 chars)

Calculator By Hasnine Kabir
=====
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit
Enter your choice (1-5): 1
Enter 1st number: 14
Enter 2nd number: 17
Result: 31
```

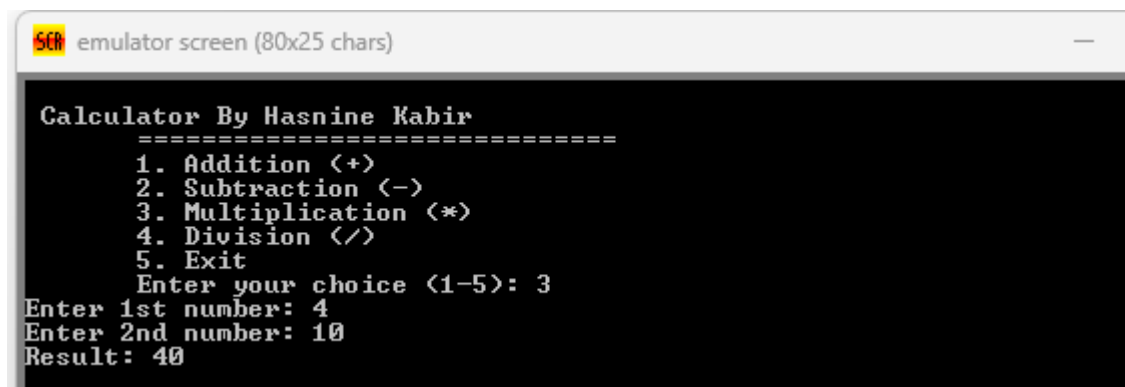
Example 2: Subtraction



```
emulator screen (80x25 chars)

Calculator By Hasnine Kabir
=====
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit
Enter your choice (1-5): 2
Enter 1st number: 10
Enter 2nd number: 6
Result: 4
```

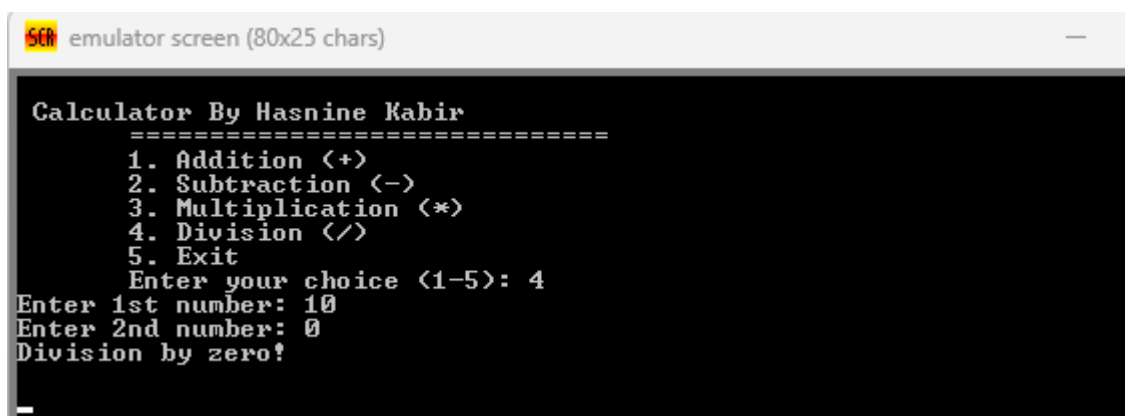
Example 3: Multiplication



```
emulator screen (80x25 chars)

Calculator By Hasnine Kabir
=====
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit
Enter your choice (1-5): 3
Enter 1st number: 4
Enter 2nd number: 10
Result: 40
```

Example 4: Divide by zero



```
emulator screen (80x25 chars)

Calculator By Hasnine Kabir
=====
1. Addition (+)
2. Subtraction (-)
3. Multiplication (*)
4. Division (/)
5. Exit
Enter your choice (1-5): 4
Enter 1st number: 10
Enter 2nd number: 0
Division by zero!
```

Conclusion

This project successfully demonstrates how a simple calculator can be built using structured programming concepts. By implementing input handling, arithmetic operations, modular functions, and output formatting, the project showcases fundamental skills required for larger software development tasks.

The calculator is easy to use, efficient, and organized. It highlights the importance of:

- Clean user interface design
- Proper input validation
- Modular code structure
- Error handling in computation